

编译原理

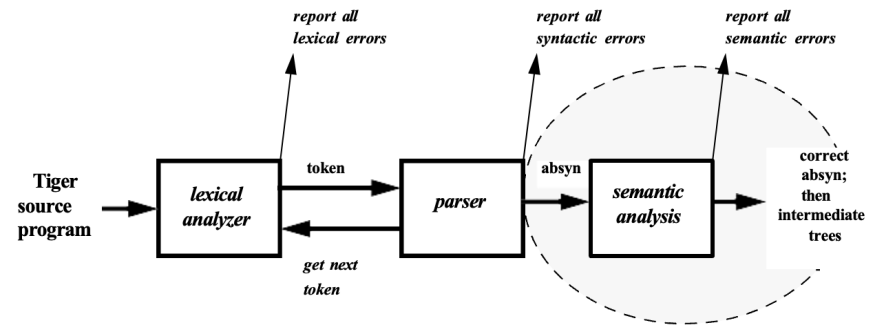
5. 语义分析

rainoftime.github.io

**浙江大学
计算机科学与技术学院**

Content

1. Introduction
2. Lexical Analysis
3. Parsing
4. Abstract Syntax
- 5. Semantic Analysis**
6. Activation Record
7. Translating into Intermediate Code
8. Basic Blocks and Traces
9. Instruction Selection
10. Liveness Analysis
11. Register Allocation
13. Garbage Collection
14. Object-oriented Languages
18. Loop Optimizations



The Limitations of CFG and Parsing

- Does the corresponding parser accept the programs?

<pre>S ::= Decl Stmt Decl ::= Type id Decl; Decl Type ::= string int Stmt ::= Stmt; Stmt id = Exp ... Exp ::= Exp * Exp id num char* ...</pre>

<pre>string x; int z; x= "hello world"; z=x+1;</pre>	<pre>int x; int z; z=x+1;</pre>
---	----------------------------------

- Many things can not be decided by syntax analysis
 - Is an array access out of bound?
 - Memory layout information for data structures
 - ...

(广义的)Semantic Analysis

- Analyze the semantics of programs (usually) through various **program representations**
 - Control flow graph (CFG)
 - Static Single Assignment (SSA)
 - Program dependence graph (PDG)
 - ...
- Sample semantic analysis
 - Type checking
 - Dead code elimination
 - Code generation.. etc

(狭义的)Semantic Analysis

- 编译原理相关课程默认的语义分析
- **Name resolution** — Connect each variable *use* to its *declaration*
- **Type checking** — Verify each expression has the correct type
- **Scope management** — Honour scoping rules (block structure, nesting)
- **Translation** — Lower the AST into a simpler IR suitable for code generation

Outline



Symbol Table



Type Checking

1. Symbol Table

- **Symbol Table**
- **Efficient Implementation**

Symbol Table

- **Binding:** give a symbol a meaning, denotes by $\mapsto$

Name/Symbol	Meaning/Attribute
variable identifier	type, value, access info, ...
type identifier	type (e.g., int, string)
function identifier	args.&result type, ...

- **Environment:** a set of bindings
 - E.g., an environment $\{g \mapsto \text{string}, a \mapsto \text{int}\}$
- **Symbol table:** the implementation of environment

Semantic analysis: traverse the abstract syntax tree (AST) in certain order while maintaining a symbol table

Example of Symbol Table

- Suppose at the very beginning the environment is σ_0

1.	function f(a:int, b:int, c:int) =	$\sigma_1 = \sigma_0 + \{a \mapsto \text{int}, b \mapsto \text{int}, c \mapsto \text{int}\}$
2.	(print_int(a+c) ;	
3.	let var j := a+b	$\sigma_2 = \sigma_1 + \{j \mapsto \text{int}\}$
4.	var a := "hello"	
5.	in print(a); print_int(j)	
6.	end ;	
7.	print_int(b)	
8.	)	

- In line2, the identifiers **a**, **c** can be looked up in σ_1
- In lines 3, the environment is updated

Example of Symbol Table

- Each local variable has a *scope* in which it is **visible**.

1.	function f(a:int, b:int, c:int) =	$\sigma_1 = \sigma_0 + \{a \mapsto \text{int}, b \mapsto \text{int}, c \mapsto \text{int}\}$
2.	(print_int(a+c) ;	
3.	let var j := a+b	$\sigma_2 = \sigma_1 + \{j \mapsto \text{int}\}$
4.	var a := "hello"	$\sigma_3 = \sigma_2 + \{a \mapsto \text{string}\}$
5.	in print(a); print_int(j)	
6.	end ;	
7.	print_int(b)	
8.	)	

- Let's consider line 4: $\sigma_3 = \sigma_2 + \{a \mapsto \text{string}\}$
 - σ_2 contains $a \mapsto \text{int}$. So, what is the binding of **a** in σ_3 ?
 - 在 σ_3 中, $\{a \mapsto \text{string}\}$ **覆盖**了 σ_2 中a的binding!

约定: Bindings in the right-hand table override those in the left

Example of Symbol Table

- Each local variable has a *scope* in which it is **visible**.

1.	function f(a:int, b:int, c:int) =	$\sigma_1 = \sigma_0 + \{a \mapsto \text{int}, b \mapsto \text{int}, c \mapsto \text{int}\}$
2.	(print_int(a+c) ;	
3.	let var j := a+b	$\sigma_2 = \sigma_1 + \{j \mapsto \text{int}\}$
4.	var a := "hello"	$\sigma_3 = \sigma_2 + \{a \mapsto \text{string}\}$
5.	in print(a); print_int(j)	
6.	end ;	σ_2, σ_3 都被 “丢弃”
7.	print_int(b)	
8.	)	

- As the semantic analysis reaches **the end of each scope**, the identifier bindings local to that scope are discarded.
 - In line 6, σ_3 is discarded and σ_1 is back

Example of Symbol Table: Summary

```
1 function f(a:int, b:int, c:int) =  
2   (print_int(a+c);  
3   let var j := a+b  
4     var a := "hello"  /* Shadows parameter 'a' */  
5   in print(a); print_int(j)  
6 end;  
7 print_int(b)  
8 )
```

Line	Code	Environment Change	Current Environment
1	function f(a:int, b:int, c:int)	Add function parameters	$\sigma_1 = \sigma_0 + \{a \mapsto \text{int}, b \mapsto \text{int}, c \mapsto \text{int}\}$
2	print_int(a+c);	Use environment σ_1	σ_1
3	let var j := a+b	Add variable j	$\sigma_2 = \sigma_1 + \{j \mapsto \text{int}\}$
4	var a := "hello"	Add new variable a (shadows parameter)	$\sigma_3 = \sigma_2 + \{a \mapsto \text{string}\}$
5	print(a); print_int(j)	Use environment σ_3	σ_3
6	end;	Exit let scope	σ_1
7	print_int(b)	Use environment σ_1	σ_1
8	)	Exit function scope	σ_0

Multiple Symbol Tables

- In some languages there can be several active environments at once: Each module, or class, or record in the program has a **symbol table** σ of its own.

```
package M;  
class E {  
    static int a = 5;  
}  
class N {  
    static int b = 10;  
    static int a = E.a + b;  
}  
class D {  
    static int d = E.a + N.a;  
}
```

Java

```
 $\sigma 1 = \{ a \mapsto \text{int} \}$   
 $\sigma 2 = \{ E \mapsto \sigma 1 \}$   
 $\sigma 3 = \{ b \mapsto \text{int}, a \mapsto \text{int} \}$   
 $\sigma 4 = \{ N \mapsto \sigma 3 \}$   
 $\sigma 5 = \{ d \mapsto \text{int} \}$   
 $\sigma 6 = \{ D \mapsto \sigma 5 \}$   
 $\sigma 7 = \sigma 2 + \sigma 4 + \sigma 6$ 
```

In Java, forward reference is allowed. so E , N , and D are all compiled in the environment $\sigma 7$. The result of the analysis is $\{M \mapsto \sigma 7\}$.

1. Symbol Table

- **Symbol Table**
- **Efficient Implementation**

Interfaces of Symbol Tables

- **insert**: 将名称绑定到相关信息(如类型), 如果名称已在符号表中定义, 则新的绑定优先于旧的绑定
- **lookup**: 在符号表中查找特定名称绑定的信息
- **beginScope**: 进入一个新的作用域
- **endScope**: 退出作用域, 将符号表恢复到进入之前的状态

Implementing Symbol Tables

$$\sigma_2 = \sigma_1 + \{j \mapsto \text{int}\}$$

- **命令式风格 Imperative Style**

- 直接**修改** σ_1 使其变为 σ_2 (就地更新)
- σ_2 存在期间无法访问 σ_1
- 退出作用域时, 利用辅助信息**撤销**修改, 重建 σ_1

有了新的就看不到老的, 但是退出scope的时候还能回得去

- **函数式风格 Functional Style**

- 实现 $\sigma_2 = \sigma_1 + \{x \mapsto t\}$ 时, **创建新表** σ_2 , σ_1 保持不变
- σ_1 与 σ_2 在内存中**共享大量**节点
- 退出作用域时, 直接恢复指向旧表的引用

每次发生变化的时候老的都保留着

Imperative Symbol Tables: Different Strategies

1. A stack of hash tables

- Each scope gets its own hash table
- Lookups search from most recent scope backward
- Simple but can be memory-intensive

2. A single hash table + scope markers

- One unified table with markers to track scope boundaries
- Memory efficient but requires cleanup when exiting scopes

3. Flat namespace with name mangling

- Encode scope information directly in symbol names
- No explicit scope tracking needed

• ...

重点介绍策略 ②： 单一哈希表 + 作用域标记栈，是虎书中采用的方法。

Single Hash Table + Stack of Scope Markers

- **哈希表(bindings):** A single hash table storing all active bindings across all scopes
 - hash map: $\text{id} \rightarrow \text{list of values}$ (outermost to innermost). *Last element = currently visible.*
- **辅助栈(stack):** Keep track of which symbols were added in each scope

操作	行为
<code>add(id, val)</code>	压栈 id; 向 <code>bindings[id]</code> 尾追加 <code>val</code>
<code>find(id)</code>	返回 <code>bindings[id]</code> 的最后一项
<code>beginScope()</code>	压栈 <code>ScopeMarker</code>
<code>endScope()</code>	弹出至 <code>ScopeMarker</code> , 逐个撤销绑定

```

from dataclasses import dataclass
from typing import TypeVar, Generic, Optional
T = TypeVar("T")

```

```

@dataclass
class ScopeMarker:
    """标记作用域的开始"""
    pass

```

```

class SymbolTable(Generic[T]):

```

```

    def __init__(self):
        # self.stack: 用于跟踪所有绑定和作用域的顺序的列表。包含两种元素：
        # 1. 字符串标识符 - 表示在当前作用域中添加的变量
        # 2. ScopeMarker对象 - 表示新作用域的开始
        # 元素按添加顺序排列，最新的在列表末尾
        self.stack = []
        # self.bindings: 存储所有变量绑定的字典
        # 键：变量标识符(字符串)。值：该变量所有绑定值的列表，按作用域从外到内(旧到新)排序
        # 例如：{"x": [1, 2, 3]} 表示x在最外层作用域绑定为1，中间为2，当前作用域绑定为3
        self.bindings = {}

```

Stack存标识符和
ScopeMarker

正好实现了最内层作用域的变量覆盖外层同名变量的规则

```

    def add(self, identifier: str, value: T):
        """添加新的绑定到当前作用域"""
        self.stack.append(identifier)
        self.bindings[identifier] = self.bindings.get(identifier, []) + [value]

```

```

    def find(self, identifier: str) -> Optional[T]:
        """查找给定标识符的最新绑定"""
        if identifier in self.bindings and self.bindings[identifier]:
            return self.bindings[identifier][-1]
        return None

```

```

from dataclasses import dataclass
from typing import TypeVar, Generic, Optional
T = TypeVar("T")

```

```

@dataclass
class ScopeMarker:
    """标记作用域的开始"""
    pass

```

```

class SymbolTable(Generic[T]):
    def __init__(self):
        # self.stack: 用于跟踪所有绑定和作用域的顺序的列表。包含两种元素:
        # 1. 字符串标识符 - 表示在当前作用域中添加的变量
        # 2. ScopeMarker对象 - 表示新作用域的开始
        # 元素按添加顺序排列, 最新的在列表末尾
        self.stack = []
        # self.bindings: 存储所有变量绑定的字典
        # 键: 变量标识符(字符串)。值: 该变量所有绑定值的列表, 按作用域从外到内(旧到新)排序
        # 例如: {"x": [1, 2, 3]} 表示x在最外层作用域绑定为1, 中间为2, 当前作用域绑定为3
        self.bindings = {}

```

Stack存标识符和
ScopeMarker

正好实现了最内层作
用域的变量覆盖外层
同名变量的规则

```

def add(self, identifier: str, value: T):
    """添加新的绑定到当前作用域"""
    self.stack.append(identifier)
    self.bindings[identifier] = self.bindings.get(identifier, []) + [value]

```

1. 将标识符添加到栈中, 记录添加顺序
2. 在bindings中为该标识符添加新的绑定值

```

def find(self, identifier: str) -> Optional[T]:
    """查找给定标识符的最新绑定"""
    if identifier in self.bindings and self.bindings[identifier]:
        return self.bindings[identifier][-1]
    return None

```

如果标识符存在且有绑定值,
返回最后一个(最新的)绑定

```
def begin_scope(self):
```

```
    """开始一个新的作用域"""
```

```
    self.stack.append(ScopeMarker())
```

1. 向栈中添加一个作用域标记
2. 标记将作为当前作用域的边界

```
def end_scope(self):
```

```
    """结束当前作用域，移除该作用域中的所有绑定
```

```
    工作原理是从栈顶开始弹出元素，直到遇到一个作用域标记(ScopeMarker)
```

- 对于每个弹出的标识符，移除其最新的绑定
- 最后移除作用域标记本身

```
    """
```

```
    # 处理当前作用域中的所有标识符
```

```
    while len(self.stack) > 0 and not isinstance(self.stack[-1], ScopeMarker):
```

```
        # 移除该标识符最新(最内层作用域)的绑定
```

```
        # bindings[identifier]是一个列表，[-1]索引指向最后添加的值
```

```
        identifier = self.stack.pop()
```

```
        self.bindings[identifier].pop()
```

```
        # 如果该标识符没有剩余绑定，从字典中删除它
```

```
        if not self.bindings[identifier]:
```

```
            self.bindings.pop(identifier)
```

```
    # 处理完所有标识符后，如果栈不为空，弹出作用域标记本身
```

```
    # 这表示当前作用域现已完全关闭
```

```
    if self.stack:
```

```
        self.stack.pop()
```

```
def begin_scope(self):  
    """开始一个新的作用域"""  
    self.stack.append(ScopeMarker())
```

```
def end_scope(self):  
    """结束当前作用域，移除该作用域中的所有绑定
```

工作原理是从栈顶开始弹出元素，直到遇到一个作用域标记(ScopeMarker)

- 对于每个弹出的标识符，移除其最新的绑定
- 最后移除作用域标记本身

```
"""
```

```
# 处理当前作用域中的所有标识符
```

```
while len(self.stack) > 0 and not isinstance(self.stack[-1], ScopeMarker):
```

```
    # 移除该标识符最新(最内层作用域)的绑定
```

```
    # bindings[identifier]是一个列表，[-1]索引指向最后添加的值
```

```
    identifier = self.stack.pop()
```

```
    self.bindings[identifier].pop()
```

```
    # 如果该标识符没有剩余绑定，从字典中删除它
```

```
    if not self.bindings[identifier]:
```

```
        self.bindings.pop(identifier)
```

```
# 处理完所有标识符后，如果栈不为空，弹出作用域标记本身
```

```
# 这表示当前作用域现已完全关闭
```

```
if self.stack:
```

```
    self.stack.pop()
```

1. 从栈顶开始，弹出所有标识符，直到遇到作用域标记

2. 对每个弹出的标识符，移除其最新的绑定

- 如果某个标识符没有剩余绑定，将其从bindings字典中移除

3. 最后弹出作用域标记本身

Example: Imperative Symbol Tables

Step 0: 初始状态：创建空符号表，栈与绑定表均为空。

创建符号表实例

```
st = SymbolTable[int]()
```

步骤1: 全局作用域

```
st.add("x", 10)
```

```
st.add("y", 20)
```

步骤2: 进入嵌套作用域

```
st.begin_scope()
```

```
st.add("z", 30)
```

在新作用域中重新定义x

```
st.add("x", 100)
```

步骤3: 退出嵌套作用域

```
st.end_scope()
```



Stack (辅助栈)

(空栈)



Bindings (绑定表)

(空)

Example: Imperative Symbol Tables

Step 1: `add("x", 10)`: 将 "x" 压入栈, `bindings["x"] = [10]`

```
# 创建符号表实例
st = SymbolTable[int]()
```

```
# 步骤1: 全局作用域
```

```
st.add("x", 10)
```

```
st.add("y", 20)
```

```
# 步骤2: 进入嵌套作用域
```

```
st.begin_scope()
```

```
st.add("z", 30)
```

```
# 在新作用域中重新定义x
```

```
st.add("x", 100)
```

```
# 步骤3: 退出嵌套作用域
```

```
st.end_scope()
```

 **Stack** (辅助栈)
[0] x ← TOP

 **Bindings** (绑定表)
x ↦ [10]

Example: Imperative Symbol Tables

Step 2: `add("y", 20)`: 将 "y" 压入栈, `bindings["y"] = [20]`。

```
# 创建符号表实例
st = SymbolTable[int]()
```

```
# 步骤1: 全局作用域
st.add("x", 10)
st.add("y", 20)
```

```
# 步骤2: 进入嵌套作用域
st.begin_scope()
st.add("z", 30)
# 在新作用域中重新定义x
st.add("x", 100)
```

```
# 步骤3: 退出嵌套作用域
st.end_scope()
```

 **Stack** (辅助栈)

```
[1] y ← TOP
[0] x
```

 **Bindings** (绑定表)

```
x ↦ [10]
y ↦ [20]
```

Example: Imperative Symbol Tables

Step 3: `begin_scope()`: 压入 `ScopeMarker`, 标记新作用域的开始。

```
# 创建符号表实例
st = SymbolTable[int]()
```

```
# 步骤1: 全局作用域
st.add("x", 10)
st.add("y", 20)
```

```
# 步骤2: 进入嵌套作用域
st.begin_scope()
st.add("z", 30)
# 在新作用域中重新定义x
st.add("x", 100)
```

```
# 步骤3: 退出嵌套作用域
st.end_scope()
```

 **Stack** (辅助栈)

```
[2]  Marker ← TOP
[1] y
[0] x
```

 **Bindings** (绑定表)

```
x ↦ [10]
y ↦ [20]
```

Example: Imperative Symbol Tables

Step 4: `add("z", 30)`: 将 "z" 压入栈, `bindings["z"] = [30]`。

```
# 创建符号表实例
st = SymbolTable[int]()
```

```
# 步骤1: 全局作用域
st.add("x", 10)
st.add("y", 20)
```

```
# 步骤2: 进入嵌套作用域
st.begin_scope()
```

```
st.add("z", 30)
```

```
# 在新作用域中重新定义x
st.add("x", 100)
```

```
# 步骤3: 退出嵌套作用域
st.end_scope()
```

 **Stack** (辅助栈)

```
[3] z ← TOP
[2] ◆ Marker
[1] y
[0] x
```

 **Bindings** (绑定表)

```
x ↦ [10]
y ↦ [20]
z ↦ [30]
```

Example: Imperative Symbol Tables

Step 5: add("x", 100): "x" 再次入栈;

bindings["x"] = [10, 100], find("x") 返回最新值 100（内层遮蔽外层）。

创建符号表实例

```
st = SymbolTable[int]()
```

步骤1: 全局作用域

```
st.add("x", 10)
```

```
st.add("y", 20)
```

步骤2: 进入嵌套作用域

```
st.begin_scope()
```

```
st.add("z", 30)
```

在新作用域中重新定义x

```
st.add("x", 100)
```

步骤3: 退出嵌套作用域

```
st.end_scope()
```

 **Stack** (辅助栈)

```
[4] x ← TOP
[3] z
[2] ◆ Marker
[1] y
[0] x
```

 **Bindings** (绑定表)

```
x ↦ [10, 100]
y ↦ [20]
z ↦ [30]
```

Example: Imperative Symbol Tables

Step 6: end_scope(): 弹出至 ScopeMarker —
依次撤销 $x=100$ 和 $z=30$ ，最后弹出标记。恢复至全局作用域。

```
# 创建符号表实例  
st = SymbolTable[int]()
```

```
# 步骤1: 全局作用域  
st.add("x", 10)  
st.add("y", 20)
```

```
# 步骤2: 进入嵌套作用域  
st.begin_scope()  
st.add("z", 30)  
# 在新作用域中重新定义x  
st.add("x", 100)
```

```
# 步骤3: 退出嵌套作用域  
st.end_scope()
```

 **Stack** (辅助栈)

```
[1] y ← TOP  
[0] x
```

 **Bindings** (绑定表)

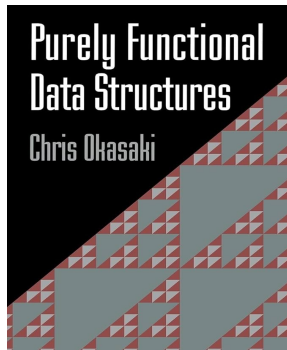
```
x ↦ [10]  
y ↦ [20]
```

Functional Symbol Tables

- Basic idea the approach:
 - When implementing $\sigma2 = \sigma1 + \{x \mapsto t\}$
 - Creating a **new** table $\sigma2$, instead of modifying $\sigma1$
 - When deleting, restore to the old table
(“方便快捷”回退”)

Typically implemented with balanced trees

- 函数式编程中的“不可变数据结构”思想

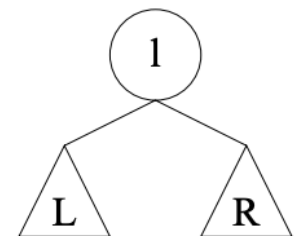


https://en.wikipedia.org/wiki/Purely_functional_data_structure

<https://xavierleroy.org/CdF/2022-2023/1.pdf>

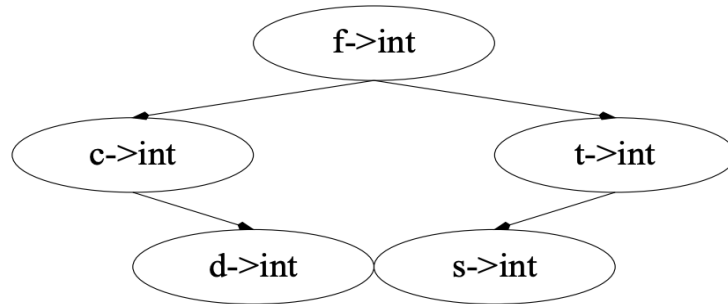
Functional Symbol Tables

- Basic idea the approach:
 - When implementing $\sigma_2 = \sigma_1 + \{x \mapsto t\}$
 - Creating a **new** table σ_2 , instead of modifying σ_1
(σ_1 and σ_2 share many data in the memory)
 - When deleting, restore to the old table
- Typically, implemented via binary search trees (BSTs)
 - Each node contains (key, binding)
 - Use string comparison for “less than” ordering.
 - BST invariant:
 - For all nodes $n \in L$, $\text{key}(n) < \text{key}(l)$
 - For all nodes $n \in R$, $\text{key}(n) \geq \text{key}(l)$



BST-based Functional Symbol Tables

- **Lookup** : $O(\log(n))$ for a balanced tree of n nodes.



遵循标准BST搜索算法：与当前节点比较，小则向左，大则向右

lookup(tree, key):

if tree is empty: return NOT_FOUND

if key < tree.key: return lookup(tree.left, key)

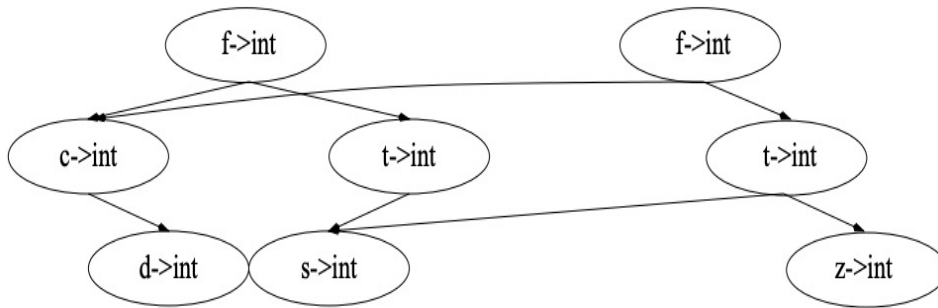
if key > tree.key: return lookup(tree.right, key)

return tree.binding // key == tree.key

- **Time complexity:** $O(\log n)$ for a balanced tree of n bindings.
- Worst-case $O(n)$ if unbalanced — mitigated with AVL trees, etc.

BST-based Functional Symbol Tables

- **Insert:** Copy the nodes from the root to the parent of the inserted node.
 - insert $z \rightarrow \text{int}$, create node z , copy all ancestors of z



使用路径复制技术：“复制从根节点到被插入节点的父节点的所有节点”

效果: 避免完整拷贝所有旧版本

Only **$O(\log n)$** nodes are **duplicated** (the ancestors along the insertion path). All other nodes are *shared* between old & new trees.

Summary: Implementation of Symbol Tables

- **Imperative Style : (side effects)**
 - One mutable table, updated **in-place**.
 - **Enter scope:** insert new bindings; save undo info.
 - **Exit scope:** undo previous additions to restore old table.
 - Typically uses a hash table + auxiliary undo stack.
- **Functional Style : (no side effects)**
 - Each scope produces a **new** environment.
 - **Enter scope:** create new env by adding to old (old env intact).
 - **Exit scope:** simply switch back to old env pointer.
 - Typically uses a balanced BST with path copying.
 - Lookup & insert $O(\log n)$; **all environments coexist**.

Imperative vs. Functional Symbol Tables

Criterion	Imperative	Functional
Data Structures	Standard dictionaries/hash	Persistent data structures
Need to revisit old scopes?	❌ Difficult (need snapshots)	✅ All versions kept
Lookup speed	$O(1)$ expected (hash table)	$O(\log n)$
Insert speed	$O(1)$ amortised	$O(\log n)$
Scope exit	$O(k)$ — undo k bindings	$O(1)$ — swap pointer
Memory model	Destructive update	Persistent / shared
Concurrency	Requires explicit synchronization	Naturally thread-safe
Debugging	Can be harder (state changes)	Easier (immutable state)

An Implementation Trick: String $\Leftrightarrow$ Symbol

- **Problem:** Hash table lookups compare strings character by character – slow!
- **Solution:** use *symbol data structure* instead
 - Each symbol object associated with *integer value*
 - All occurrences of same string map onto same symbol
(2 different strings map onto different symbols)

2. Type Checking

- **类型及其作用**
- Tiger类型系统
- Tiger类型检查

类型在编程语言中的作用

- **开发效率**：更高层次的编程抽象, e.g.,
 - Intersection Type、Dependent Type
 - hoogle利用类型信息搜索API
- **运行性能**：类型指导的编译优化, e.g.,
 - 静态类型绑定避免运行时检查
 - 利用类型信息优化内存布局
- **安全可靠**：内存安全乃至功能正确, e.g.,
 - Rust线性类型保障内存安全
 - LiquidHaskell精化类型检查功能正确



编得快



跑得快



信得过

类型签名 `add_two (x : int | x > 0) → x + 2`

An Overview of Types

- The **type** of an expression tells us the values it can denote and the operations that can be applied to it.
- **Type system** --- definition of well-formed types + a set of typing rules
- **Type checking** ensures that the operations in a program are applied properly
- Static vs. dynamic type checking

Soundness and Completeness (供参考)

- **Soundness:** If the type system accepts a program, then the program will not encounter type errors at runtime

A closely related notion is “**Type safety**”:

“Well-typed programs don’t “go wrong”--Robin Milner, 1978.

- **Completeness:** If a program would not encounter type errors at runtime, then the type system accepts it

“Advanced/Modern” Type Systems (供参考)

Both brand new ideas and old research ideas that are making their way into new programming languages

Safety

Ownership Types

Lifetimes

Region-based Memory Management

Linear Types

Refinement Type

Expressiveness

Union Types

Intersection Types

Algebraic Effects

Polymorphic Effect Systems

Powerful Type Inference

Main Issues in Type Checking

- **What are valid type expressions ?**
 - e.g., int, string, unit, nil, array of int, record ...
- **How to define two types are equivalent ?**
 - 比如 , 两个record类型是否相同?
- **What are the typing rules ?**
- **How much type info should be specified in the source program?**
 - implicitly-typed lang., e.g., ML ----- uses type inference

2. Type Checking

- 类型及其作用
- Tiger类型系统
- Tiger类型检查

Environments for Type Checking

- Tiger语义分析需要维护2个环境:

1. **Type** environment

将类型符号映射到其代表的类型

2. **Value** environment

变量符号映射到其类型；将函数符号映射到参数与返回类型

```
let type a = int  
  var a := 1  
in ...  
end
```

Both a's can
be used

```
let function a (b: int) = ...  
  var a := 1  
in ...  
end
```

var a hides
function a

Types in the Tiger Language

- **The primitive type:** `int`, `string`
- **The constructed type:** `records` and `arrays` from other types
- **Name Type:** User-defined types via type declarations
- **NIL type** (for records)

Type kind	Tiger syntax	Example
Integer	<code>int</code>	<code>42</code>
String	<code>string</code>	<code>"hello"</code>
Record	<code>{f₁:t₁, ..., f_n:t_n}</code>	<code>point{x:int,y:int}</code>
Array	<code>array of t</code>	<code>intArray[10] of 0</code>
Named	<code>type id = ty</code>	<code>type myint = int</code>
Nil	<code>nil</code>	special record null

nil is compatible with *every* record type (used for initialisation and as a null pointer).

Example: Types in Tiger

- The primitive type: **int**, **string**
- The constructed type: **records** and **arrays** from other types
- Name Type: User-defined types via type declarations
- NIL type (for records)

$typec \rightarrow \text{type } type\text{-}id = ty$
 $ty \rightarrow type\text{-}id$
 $\rightarrow \{ \} tyfields \{ \}$
 $\rightarrow \text{array of } type\text{-}id$
 $tyfields \rightarrow \varepsilon$
 $\rightarrow id: type\text{-}id \{, id: type\text{-}id \}$

类型的文法定义

```
/* Basic types */  
var i: int := 42  
var s: string := "hello"  
  
/* Record type */  
type point = {x: int, y: int}  
var p: point := point {x=10, y=20}  
  
/* Array type */  
type intArray = array of int  
var arr: intArray := intArray [10] of 0
```

Type Equivalence (重要)

- **Name equivalence (NE)**

T1 and T2 are equivalent iff T1 and T2 are identical type names defined by the exact same **type declaration**.

- **Structure equivalence (SE)**

T1 and T2 are equivalent iff T1 and T2 are composed of the same constructors applied in the same order.

```
type point = {x : int, y : int}  
type ptr   = {x : int, y : int}
```

SE 等价 ✓ — 结构相同
NE 不等价 ✗ — 不同声明

Typing Rules in Tiger: Name Equivalence

- **Tiger uses name(nominal) equivalence**
- Every Tiger-language “record type expression” creates a new (and different) record type!

```
let type a = {x: int; y: int}  
  type b = {x: int; y: int}  
  var i : a := ...  
  var j : b := ...  
in i := j  
end
```

It is illegal in
Tiger.

```
let type a = {x: int; y: int}  
in var point = {x = 10, y = 20}  
end
```

- The type of point is an anonymous record type not equivalent to **a**.
- Each **anonymous type** gets its own unique "hidden" identity.

Typing Rules in Tiger: Name Equivalence

- **Type aliases**

```
let type a = {x: int; y: int}  
    type b = a  
    var i : a := ...  
    var j : b := ...  
in i := j  
end
```

It is legal
in Tiger.

With name equivalence, **b** is considered exactly the same type as **a** because it directly refers to the name a

Typing Rules in Tiger: Recursive Types

- Recursive types allowed through records and arrays

```
let
```

```
    type Circular = Circular
```

```
    /* 错误: 直接循环类型定义 */
```

```
    var c: Circular := nil
```

```
in
```

c Tiger不允许不通过数组或记录的
直接循环类型定义

```
end
```

```
type List = {value: int,  
next: ListRef}
```

```
type ListRef = array of  
List
```

Tiger允许通过数组或记录创建间接
递归类型, 例如链

- Every cycle in a set of mutually recursive type declarations **MUST** pass through a record or array declaration !

```
type a = b  
type b = a
```

It is illegal
in Tiger.

```
type a = b  
type b = d  
type c = a  
type d = a
```

It is illegal
in Tiger.

```
type a = b  
type b = {i: a}
```

contains an illegal cycle:
a -> b -> d -> a

For more rules, read Appendix of Tiger book

Typing Rules in Tiger: A Few More ...

- **No implicit conversions**
- All variables must be initialized at declaration
- Function call: the types of formal parameters must be equivalent to the types of actual arguments.
- Array subscript must have integer type
- The expression **nil** has the special type **NIL**.
 - **NIL belongs to every record type**
 - It is equivalent to any record type

3. Type Checking

- 类型及其作用
- Tiger类型系统
- Tiger类型检查
 - Formalization (无需掌握)
 - Type Checking

Formalizing Type Systems: Typing Context

- We write the symbol table as Γ (typing context)
 - $\Gamma = x_1:t_1, x_2:t_2, \dots, x_n:t_n$

```
{ //  $\Gamma = \emptyset$   
  int i, n; //  $\Gamma = i:int, n:int$   
  for (i = 0; i < n; i++) {  
    boolean b = true; //  $\Gamma = i:int, n:int, b:boolean$   
  
    ...  
  
  } //  $\Gamma = i:int, n:int$   
} //  $\Gamma = \emptyset$ 
```

Formalizing Type System: Typing Rules

- **Typing judgments:** statements about the types of expressions ($\Gamma \vdash e : \tau$)
- **Typing rule:** Each typing rule takes the form of

premise1, premise2, ..., premiseN

conclusion

"If all premises are true, then the conclusion is true."

$\Sigma \vdash P1 : T1 \quad \dots \quad \Sigma \vdash Pn : Tn$

$\Sigma \vdash C : T$

Some Typing Rules for Tiger (无需掌握)

- Basic Expression Typing Rules

$$\frac{n \text{ is an integer literal}}{\Gamma \vdash n : \text{int}} \quad (T\text{-Int})$$

$$\frac{s \text{ is a string literal}}{\Gamma \vdash s : \text{string}} \quad (T\text{-String})$$

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash x : \tau} \quad (T\text{-Var})$$

$$\frac{}{\Gamma \vdash \text{nil} : \text{record}}$$

$$\frac{\Gamma \vdash e_1 : \text{int}, \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 + e_2 : \text{int}} \quad (T\text{-Plus})$$

$$\frac{\Gamma \vdash e_1 : \text{int}, \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 - e_2 : \text{int}} \quad (T\text{-Minus})$$

$$\frac{\Gamma \vdash e_1 : \text{int}, \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 * e_2 : \text{int}} \quad (T\text{-Mul})$$

$$\frac{\Gamma \vdash e_1 : \text{int}, \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 / e_2 : \text{int}} \quad (T\text{-Div})$$

Some Typing Rules of Tiger (无需掌握)

- **Comparison and Logical Operators**

$$\frac{\Gamma \vdash e_1 : \text{int}, \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 = e_2 : \text{int}}$$

$$\frac{\Gamma \vdash e_1 : \text{string}, \Gamma \vdash e_2 : \text{string}}{\Gamma \vdash e_1 = e_2 : \text{int}}$$

$$\frac{\Gamma \vdash e_1 : \tau, \Gamma \vdash e_2 : \tau, \tau \text{ is a record type}}{\Gamma \vdash e_1 = e_2 : \text{int}}$$

$$\frac{\Gamma \vdash e_1 : \text{int}, \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 < e_2 : \text{int}}$$

- **Control Structures**

$$\frac{\Gamma \vdash e_1 : \text{int}, \Gamma \vdash e_2 : \tau, \Gamma \vdash e_3 : \tau}{\Gamma \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 : \tau}$$

$$\frac{\Gamma \vdash e_1 : \text{int}, \Gamma \vdash e_2 : \text{void}}{\Gamma \vdash \text{if } e_1 \text{ then } e_2 : \text{void}}$$

$$\frac{\Gamma \vdash e_1 : \text{int}, \Gamma \vdash e_2 : \text{void}}{\Gamma \vdash \text{while } e_1 \text{ do } e_2 : \text{void}}$$

$$\frac{\Gamma \vdash e_1 : \text{void}, \Gamma \vdash e_2 : \tau}{\Gamma \vdash (e_1 ; e_2) : \tau}$$

$$\frac{\text{Inside a loop}}{\Gamma \vdash \text{break} : \text{void}}$$

$$\frac{\Gamma \vdash e_1 : \text{int}, \Gamma \vdash e_2 : \text{int}, \Gamma + \{i \mapsto \text{int}\} \vdash e_3 : \text{void}}{\Gamma \vdash \text{for } i := e_1 \text{ to } e_2 \text{ do } e_3 : \text{void}}$$

Type Derivations (无需掌握)

- A type derivation is a proof tree showing how a typing judgment is derived from the rules.

Example: Deriving Type for $x + 2$

Assume $\Gamma = \{x:\text{int}\}$

```
x:int ∈ Γ
----- (T-Var)
Γ ⊢ x : int

----- (T-Int)
Γ ⊢ 2 : int

----- (T-Plus)
Γ ⊢ x + 2 : int
```

- Read from leaves (axioms) to root (final judgment)
- Each interior node corresponds to applying a typing rule
- The premises of a rule are the children of the node
- The conclusion is the node itself

Derivation trees mirror the recursive structure of the type checker implementation!

Example: Type Derivations (无需掌握)

- A type derivation is a proof tree showing how a typing judgment is derived from the rules.

Example: Deriving Type for $x + 2$

Assume $\Gamma = \{x:\text{int}\}$

$$\frac{x:\text{int} \in \Gamma}{\Gamma \vdash x : \text{int}} \text{ (T-Var)}$$

(T-Var): 如果变量声明在上下文中存在, 那么我们可以使用这个声明的类型。因为 $x:\text{int}$ 在上下文 Γ 中, 所以可以使用这个事实作为推导的起点。

Example: Type Derivations (无需掌握)

- A type derivation is a proof tree showing how a typing judgment is derived from the rules.

Example: Deriving Type for $x + 2$

Assume $\Gamma = \{x:\text{int}\}$

$$\frac{x:\text{int} \in \Gamma}{\Gamma \vdash x : \text{int}} \text{ (T-Var)}$$
$$\frac{}{\Gamma \vdash 2 : \text{int}} \text{ (T-Int)}$$

接下来我们处理数字 2。根据 *(T-Int)* 规则，整数常量的类型为 `int`。

Example: Type Derivations (无需掌握)

- A type derivation is a proof tree showing how a typing judgment is derived from the rules.

Example: Deriving Type for $x + 2$

Assume $\Gamma = \{x:\text{int}\}$

$$\begin{array}{c} \begin{array}{c} x:\text{int} \in \Gamma \\ \hline \Gamma \vdash x : \text{int} \end{array} \quad \text{(T-Var)} \qquad \begin{array}{c} \hline \Gamma \vdash 2 : \text{int} \end{array} \quad \text{(T-Int)} \\ \hline \Gamma \vdash x + 2 : \text{int} \quad \text{(T-Plus)} \end{array}$$

最后，我们应用 **(T-Plus)** 规则：如果 $\Gamma \vdash e_1 : \text{int}$ 且 $\Gamma \vdash e_2 : \text{int}$ ，则 $\Gamma \vdash e_1 + e_2 : \text{int}$ 。

Example: Type Derivations (无需掌握)

- A type derivation is a proof tree showing how a typing judgment is derived from the rules.

Complex Example:
if $x < 5$ then $x+1$ else $x-1$

Assume $\Gamma = \{x:\text{int}\}$

```
x:int ∈ Γ
----- (T-Var)   ----- (T-Int)
Γ ⊢ x : int       Γ ⊢ 5 : int
----- (T-Lt)
Γ ⊢ x < 5 : int
```

```
x:int ∈ Γ
----- (T-Var)   ----- (T-Int)
Γ ⊢ x : int       Γ ⊢ 1 : int
----- (T-Plus)
Γ ⊢ x + 1 : int
```

```
x:int ∈ Γ
----- (T-Var)   ----- (T-Int)
Γ ⊢ x : int       Γ ⊢ 1 : int
----- (T-Minus)
Γ ⊢ x - 1 : int
```

```
----- (T-If)
Γ ⊢ if x < 5 then x+1 else x-1 : int
```

3. Type Checking

- 类型及其作用
- Tiger类型系统
- Tiger类型检查
 - Formalization (无需掌握)
 - Type Checking

Environments for Tiger Type Checking

```
let type a = int
    var a: a := 5
    var b: a := a
in b+a
end
```

1. Type environment (Tenv)

- Maps type symbol to type that it stands for

2. Value environment (Venv)

- Maps variable symbol to its type
- Maps function symbol to parameter and result types

类型系统: 用推理规则来确定**有哪些类型**、**不同符号**(如变量、函数)**的类型是什么**

Main Idea of Type Checking

- Type checking is done **compositionally**:
 - Break down expressions into their subexpressions
 - Type-check the subexpressions
 - Ensure that the top-level compound expression can then be given a type itself

Throughout the process, environments are maintained which records the types of all variables in the expression.

Implementing the Typing Rules

- Each typing rule corresponds to a part of the type checker implementation:

$$\frac{n \text{ is an integer literal}}{\Gamma \vdash n : \text{int}} \quad (T\text{-Int})$$

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash x : \tau} \quad (T\text{-Var})$$

$$\frac{\Gamma \vdash e_1 : \text{int}, \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 + e_2 : \text{int}} \quad (T\text{-Plus})$$

$$\frac{\Gamma \vdash e_1 : \text{int}, \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 - e_2 : \text{int}} \quad (T\text{-Minus})$$

$$\frac{\Gamma \vdash e_1 : \text{int}, \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 * e_2 : \text{int}} \quad (T\text{-Mul})$$

```
if isinstance(expr, IntLiteral):  
    result = IntType()
```

```
elif isinstance(decl, VarDecl):  
    if decl.type_name not in self.type_env:  
        raise TypeError(...)  
    var_type = self.type_env[decl.type_name]  
    ...
```

```
elif isinstance(expr, BinOpExpr):  
    left_type = self.check_expr(expr.left)  
    right_type = self.check_expr(expr.right)  
    if expr.op in ['+', '-', '*', '/']:  
        if isinstance(left_type, IntType) and  
           isinstance(right_type, IntType):  
            result = IntType()  
    else: ....
```

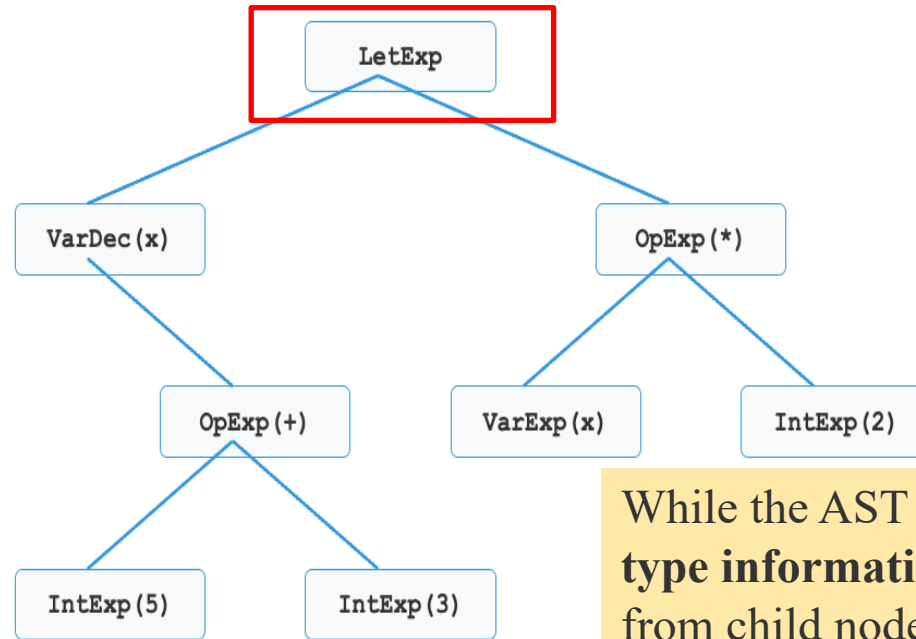
“Composing” the Implementations

Type checking is a recursive process that traverses the abstract syntax tree (AST):

1. Examine the current node's kind (IntLiteral, etc.)
2. Recursively type-check any child expressions
3. Apply the appropriate typing rule based on the node kind
4. Verify type compatibility where required
5. Report any type errors

While the AST is traversed top-down, **type information** flows bottom-up from child nodes to parent nodes:

Example: `let var x := 5 + 3 in x * 2 end`



While the AST is traversed top-down, **type information** flows bottom-up from child nodes to parent nodes:

1. Start at **LetExp** node

Begin new scopes: `venv.beginScope()`, `tenv.beginScope()`

Process declarations first (**VarDec**) before determining **LetExp**'s type

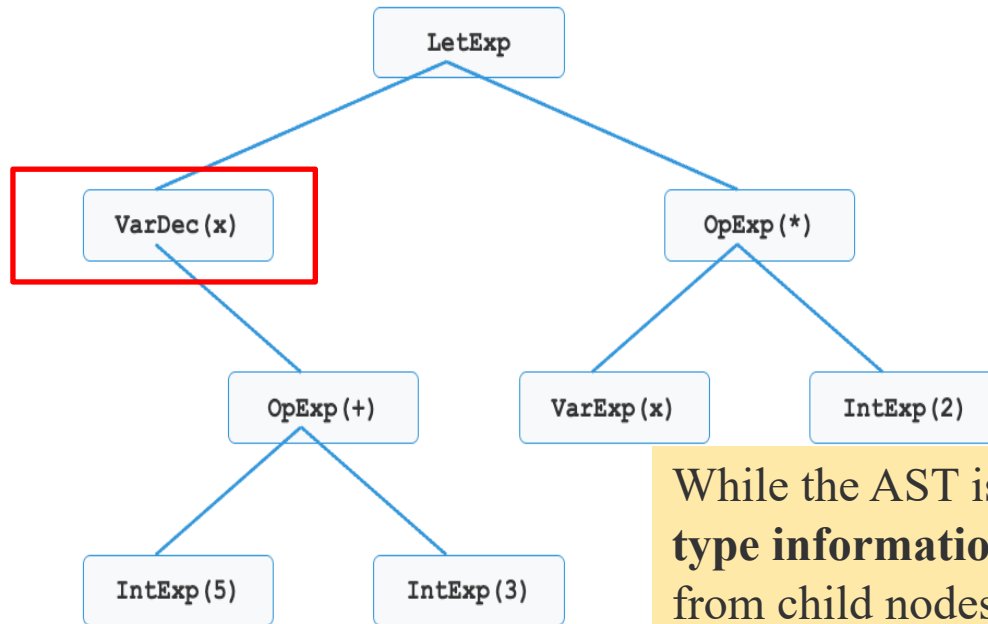
2. Process **VarDec(x)** - needs to know initializer type

Must recursively evaluate: **OpExp(+)**

3. Process **OpExp(+)** - needs operand types

Recursively evaluate: **IntExp(5)** and **IntExp(3)**

Example: `let var x := 5 + 3 in x * 2 end`



While the AST is traversed top-down, **type information** flows bottom-up from child nodes to parent nodes:

1. Start at **LetExp** node

Begin new scopes: `venv.beginScope()`, `tenv.beginScope()`

Process declarations first (**VarDec**) before determining LetExp's type

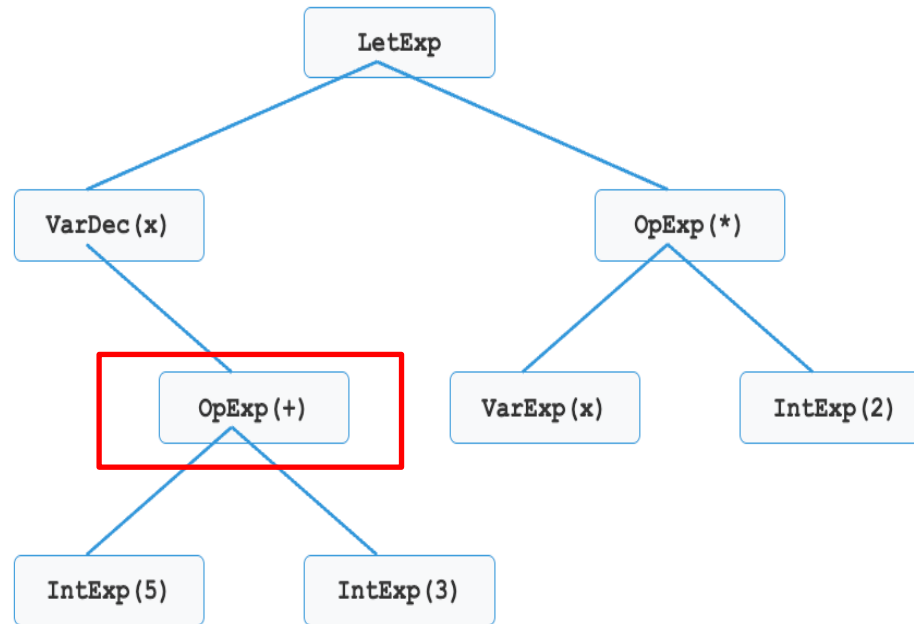
2. Process **VarDec(x)** - needs to know initializer type

Must recursively evaluate: **OpExp(+)**

3. Process **OpExp(+)** - needs operand types

Recursively evaluate: **IntExp(5)** and **IntExp(3)**

Example: `let var x := 5 + 3 in x * 2 end`



1. Start at **LetExp** node

Begin new scopes: `venv.beginScope()`, `tenv.beginScope()`

Process declarations first (**VarDec**) before determining LetExp's type

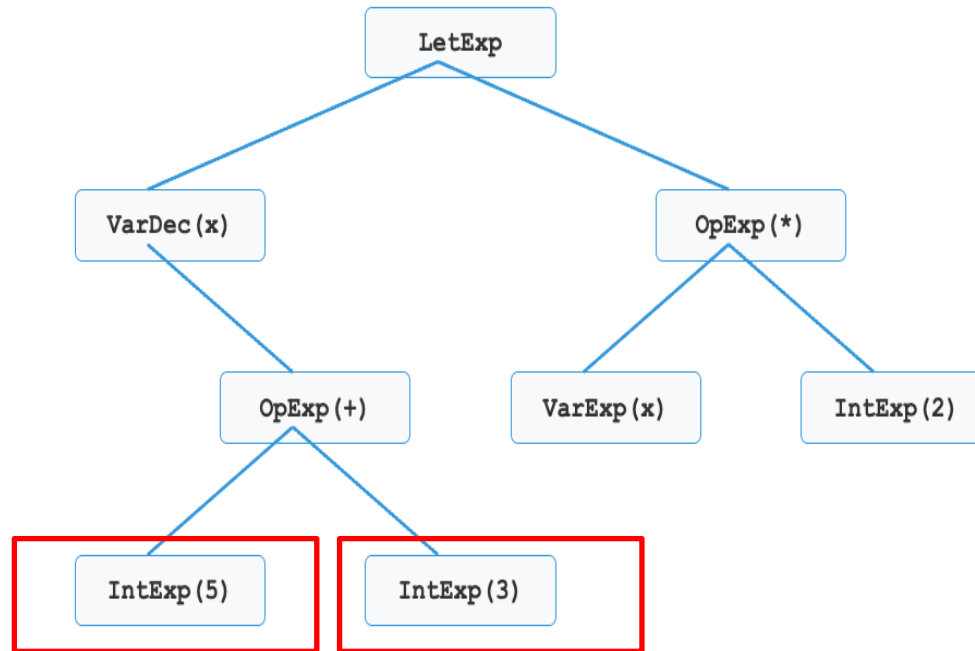
2. Process **VarDec(x)** - needs to know initializer type

Must recursively evaluate: **OpExp(+)**

3. Process **OpExp(+)** - needs operand types

Recursively evaluate: **IntExp(5)** and **IntExp(3)**

Example: `let var x := 5 + 3 in x * 2 end`



4. Evaluate **IntExp(5)** → **int** Evaluate **IntExp(3)** → **int**

$$\frac{n \text{ is an integer literal}}{\Gamma \vdash n : \text{int}} \quad (T\text{-Int})$$

5. Return to **OpExp(+)**

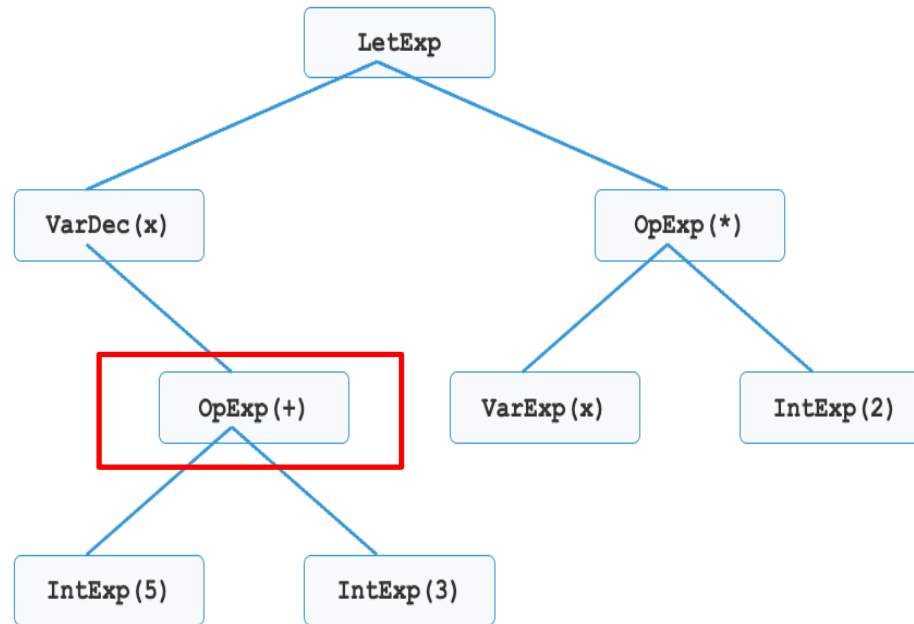
Check operand types: `int + int` → **int**

6. Return to **VarDec(x)**

Initializer has type **int**

Add to environment: `venv.enter(x, VarEntry(INT_TYPE))`

Example: `let var x := 5 + 3 in x * 2 end`



4. Evaluate **IntExp(5)** → **int** Evaluate **IntExp(3)** → **int**

5. Return to **OpExp(+)**

Check operand types: `int + int` → **int**

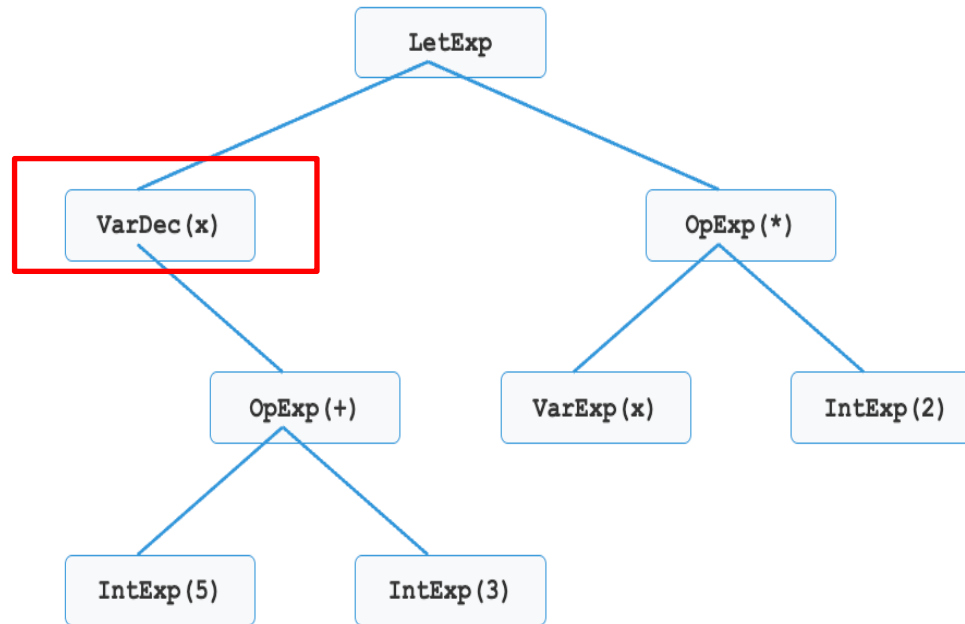
$$\frac{\Gamma \vdash e_1 : \text{int}, \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 + e_2 : \text{int}} \text{ (I-Plus)}$$

6. Return to **VarDec(x)**

Initializer has type **int**

Add to environment: `venv.enter(x, VarEntry(INT_TYPE))`

Example: `let var x := 5 + 3 in x * 2 end`



4. Evaluate **IntExp(5)** → **int** Evaluate **IntExp(3)** → **int**

5. Return to **OpExp(+)**

Check operand types: `int + int` → **int**

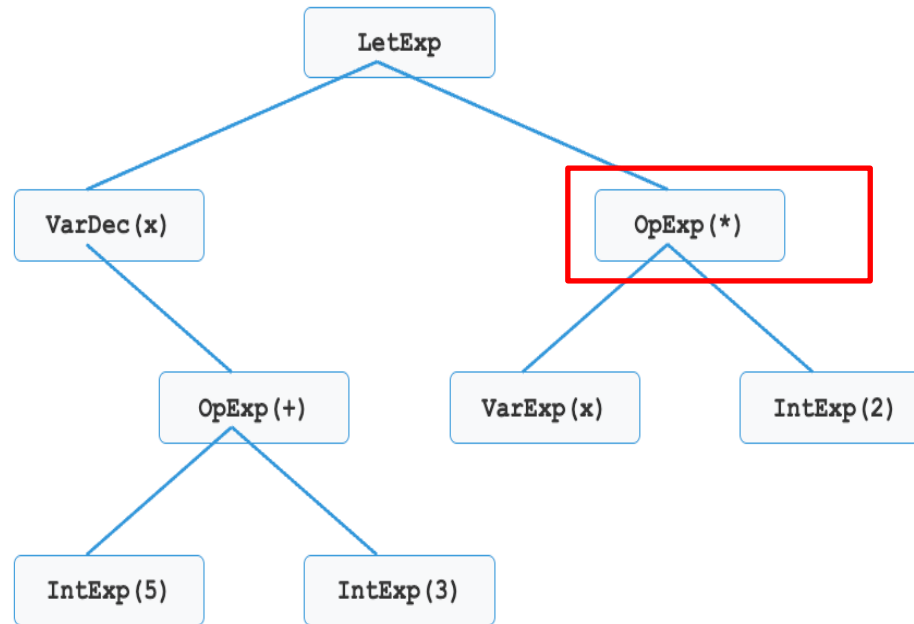
6. Return to **VarDec(x)**

Initializer has type **int**

Add to environment: `venv.enter(x, VarEntry(INT_TYPE))`

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash x : \tau} \quad (T\text{-Var})$$

Example: `let var x := 5 + 3 in x * 2 end`



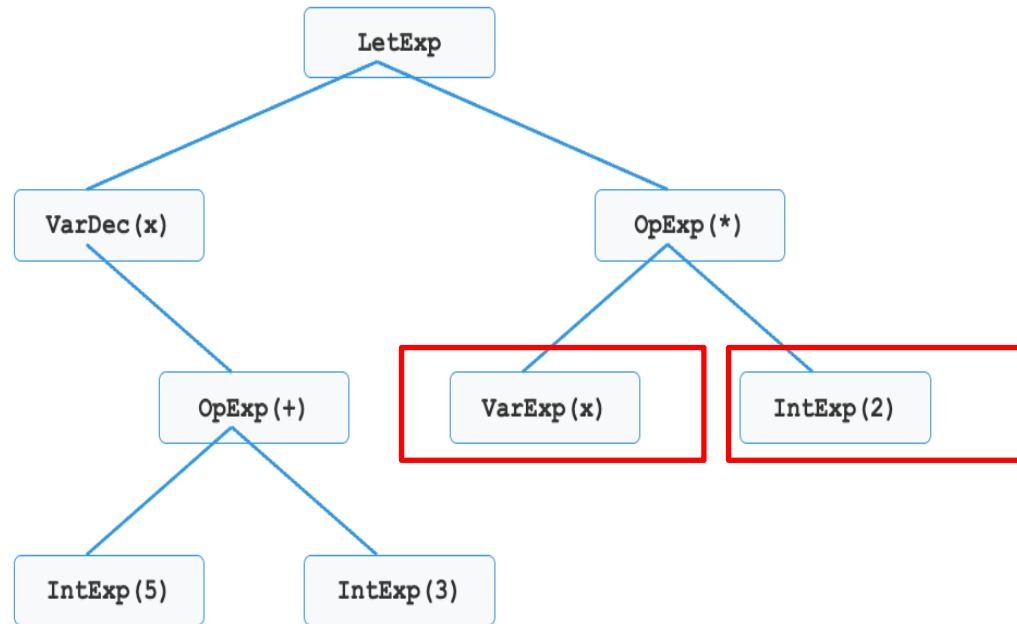
7. Now process the **LetExp** body: **OpExp(*)**
Must recursively evaluate its operands

8. Evaluate **VarExp(x)** - look up in environment → **int** Evaluate **IntExp(2)** → **int**

9. Return to **OpExp(*)**
Check operand types: `int * int` → **int**

10. Return to **LetExp**
Body has type **int**, so **LetExp** has type **int**; End scopes

Example: `let var x := 5 + 3 in x * 2 end`



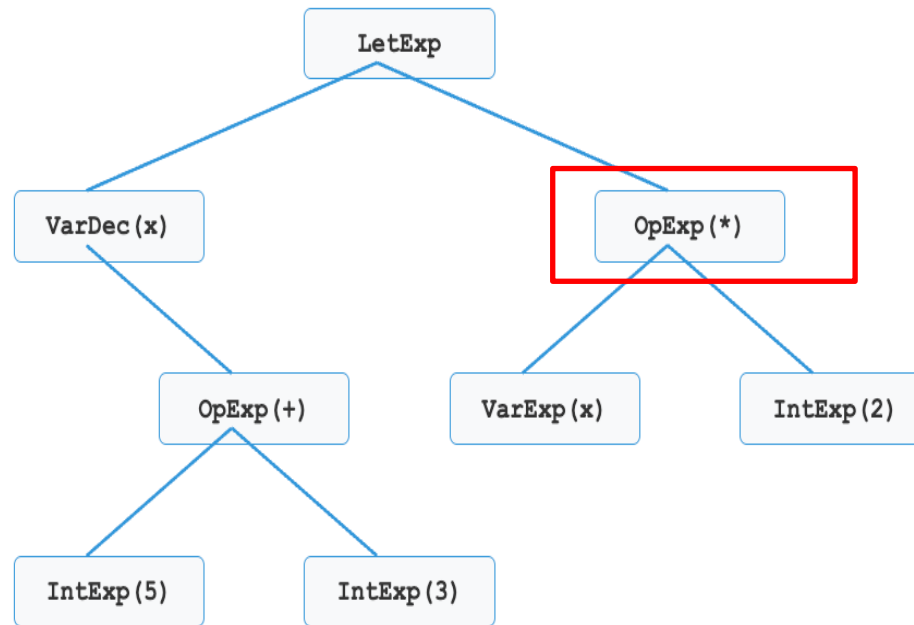
7. Now process the **LetExp** body: **OpExp(*)**
Must recursively evaluate its operands

8. Evaluate **VarExp(x)** - look up in environment → **int** Evaluate **IntExp(2)** → **int**

9. Return to **OpExp(*)**
Check operand types: `int * int` → **int**

10. Return to **LetExp**
Body has type **int**, so **LetExp** has type **int**; End scopes

Example: `let var x := 5 + 3 in x * 2 end`



7. Now process the **LetExp** body: **OpExp(*)**
Must recursively evaluate its operands

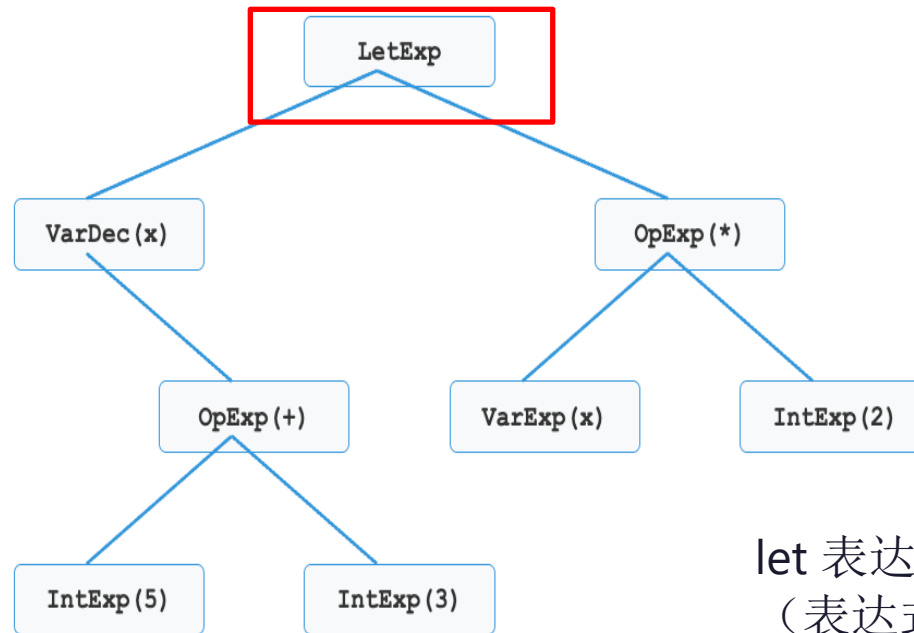
8. Evaluate **VarExp(x)** - look up in environment → **int** Evaluate **IntExp(2)** → **int**

9. Return to **OpExp(*)**
Check operand types: `int * int` → **int**

$$\frac{\Gamma \vdash e_1 : \text{int}, \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 * e_2 : \text{int}} \quad (T\text{-Mul})$$

10. Return to **LetExp**
Body has type **int**, so **LetExp** has type **int**; End scopes

Example: `let var x := 5 + 3 in x * 2 end`



let 表达式的类型 = 其 body
(表达式序列) 中最后一个
表达式的类型。

7. Now process the **LetExp** body: **OpExp(*)**
Must recursively evaluate its operands

8. Evaluate **VarExp(x)** - look up in environment → **int** Evaluate **IntExp(2)** → **int**

9. Return to **OpExp(*)**: Check operand types: `int * int` → **int**

10. Return to **LetExp**

Body has type **int**, so **LetExp** has type **int**; End scopes

Summary

- **Type Systems:** Formal sets of rules for assigning types to expressions
- **Type Checking:** Process of verifying program expressions against typing rules
- **Typing Judgments:** Formal statements about the types of expressions ($\Gamma \vdash e : \tau$)
- **Type Derivations:** Proof trees showing how types are derived from rules

Real-World Complications

- **Mutually Recursive Functions:** Functions that call each other
- **Polymorphism:** Generic types and functions
- **Subtyping:** Type hierarchies and inheritance
- **Intersection types**
- **Dependent types**
- **Gradual typing**
- ...



The Core: transExp

```
struct expty
  transVar(S_table venv,
           S_table tenv,
           A_var v);
```

```
struct expty
  transExp(S_table venv,
           S_table tenv,
           A_exp a);
```

```
void
  transDec(S_table venv,
           S_table tenv,
           A_dec d);
```

```
Ty_ty
  transTy(S_table tenv, A_ty a);
```

```
struct expty { Tr_exp exp; Ty_ty ty; }
```

Each function returns a pair:

- **exp** — the translated IR expression
- **ty** — the Tiger type of the expression

For now: we use a dummy IR —
typedef void *Tr_exp and return
NULL everywhere for exp. The real
translation comes in Chapter 7.

Focus: correctness of the **type** field.

Type-Checking: Addition (Simple Case)

```
case A_opExp: {
  A_oper oper = a->u.op.oper;
  struct expty left =
    transExp(venv,tenv,a->u.op.left);
  struct expty right =
    transExp(venv,tenv,a->u.op.right);

  if (oper == A_plusOp) {
    if (left.ty->kind != Ty_int)
      EM_error(a->u.op.left->pos,
        "integer required");

    if (right.ty->kind != Ty_int)
      EM_error(a->u.op.right->pos,
        "integer required");

    return expTy(NULL, Ty_Int());
  }
  ...
}
```

Rule for $e_1 + e_2$ in Tiger:

- 1 Recursively type-check *left* and *right* sub-expressions
- 2 Both operands **must** be integers (error if not)
- 3 Result type is **int**
Tiger has *no overloading*.

In C/Java, + works on int and float. Tiger's + is integer-only — simpler to check!

Operator Overloading — Why Tiger is Simpler

Language	$1 + 2$	$1.0 + 2$	$1.0 + 2.0$
Tiger	int ✓	type error	type error
C	int	float (implicit cast)	float
Java	int	double (widening)	double

Benefit of non-overloaded types:

No coercion rules needed, no implicit conversion, no dispatch on type. The type-checker is simpler and errors are clearer.

Real languages (C/Java):

Overloaded $+$ requires checking both types and choosing the right operation (int add vs. float add). The compiler must insert explicit conversion nodes in the IR.

Type-Checking Variables — transVar

```
struct expty transVar(  
    S_table venv, S_table tenv, A_var v) {  
    switch(v->kind) {  
        case A_simpleVar: {  
            E_enentry x =  
                S_look(venv, v->u.simple);  
            if (x && x->kind == E_varEntry)  
                return expTy(NULL,  
                    actual_ty(x->u.var.ty));  
            else {  
                EM_error(v->pos,  
                    "undefined variable %s",  
                    S_name(v->u.simple));  
                return expTy(NULL, Ty_Int());  
            }  
        }  
        case A_fieldVar: ...  
        case A_subscriptVar: ...  
    }  
}
```

For a simple variable reference:

1. Look up the identifier in **venv**
2. Check it maps to a **VarEntry** (not FunEntry)
3. Return actual_ty of its type (chase aliases)
4. If not found: report error, return Ty_Int as a *recovery type* to continue checking

Error recovery: Returning Ty_Int on error avoids cascading "undefined" errors. The compiler can continue type-checking the rest of the program.

Type-Checking Declarations

- **Declarations modify environments!**
- In Tiger, declarations appear only in a let expression.

```
struct expty transExp (S_table venv, S_table tenv, A_exp a)
{
    switch(a->kind) {
        ...
        case A_letExp: {
            struct expty exp;
            A_decList d;
            S_beginScope(venv); S_beginScope(tenv);
            for (d = a->u.let.decs; d; d=d->tail)
                transDec(venv,tenv,d->head);
            exp = transExp(venv,tenv,a->u.let.body);
            S_endScope(tenv); S_endScope(venv);
            return exp;
        }
        ...
    }
}
```

- 变量声明、类型声明、函数声明
- let中声明的可以用在in里面

Variable Declarations

- Processing a variable declaration **without a type constraint**, e.g.,

var x := exp

```
void transDec(S_table venv, S_table tenv, A_dec d) {  
    switch(d->kind) {  
        case A_varDec: {  
            struct expty e = transExp(venv, tenv, d->u.var.init);  
            S_enter(venv, d->u.var.var, E_VarEntry(e.ty));  
        }  
        ...  
    }  
    ...  
}
```

exp的类型ty(通过transExp获得)就是x的类型

Variable Declarations

- Processing a variable declaration **with a type constraint:**
 $var\ x : type-id := exp$
 - It will be necessary to check that the constraint and the initializing expression are compatible.
 - Also, initializing expressions of type **Ty_Nil** must be constrained by a **Ty_Record** type.

Type Declarations

- Nonrecursive type declarations

type type-id = ty

- transTy: translates **A_ty** to **Ty_ty** recursively

直接利用抽象语法树(AST)节点上的类型信息(**A_ty**)

```
void transDec(S_table venv, S_table tenv, A_dec d) {  
    ...  
    case A_typeDec: {  
        S_enter(tenv, d->u.type->head->name, transTy(d->u.type->head->ty));  
    }  
    ...  
}
```

NOTICE: This program fragment **only handles** the type-declaration list of length 1!

Function Declarations

- Nonrecursive function declarations

function id (tyfields) : type-id = exp

```
void transDec(S_table venv, S_table tenv, A_dec d)
{
    switch(d->kind) {
        ...
        case A_functionDec: {
            A_fundefc f = d->u.function->head;
            Ty_ty resultTy = S_look(tenv, f->result);
            Ty_tyList formalTys =
makeFormalTyList(tenv, f->params);
            S_enter(venv, f->name,
E_FunEntry(formalTys, resultTy));
            S_beginScope(venv);
            {A_fieldList l; Ty_tyList t;
                for(l=f->params, t=formalTys; l; l=l->tail,
t=t->tail)
                    S_enter(venv, l->head->name, E_VarEntry(t-
>head));
            }
            transExp(venv, tenv, d->u.function->body);
            S_endScope(venv);
            break;
        }
        ...
    }
}
```

function f(a:int):int = body

1. Look up 'int' in type env, 'int' $\mapsto$ Ty_int
2. venv' = venv + f $\mapsto$ { formals = Ty_int, result = Ty_int }
3. venv'' = venv' + a $\mapsto$ Ty_int
4. Type-check the body in {tenv, venv''} using transExp
5. Return {tenv, venv'} for use in processing expressions which refer to f

NOTICE: The above code handles very restricted functions

Function Declarations

- The above code is very stripped-down implementation:
 - it handles only the case of a single function;
 - it handles only a function with a result
 - it doesn't handle program errors
 - it doesn't check that the type of the body expression matches the declared result type
 - ...
- *function f(a: ta, b: tb) : rt = body*
- *makeFormalTyList*: traverses the list of formal parameters and returns a list of their types

Recursive Type Declarations

- How to convert the following declaration into the internal type representations ?

`type list = {first: int, rest: list}`

- **Problem:** Processing `type list = {first: int, rest: list}` requires to lookup `list` from the type environment
 - *undefined type!*
- How to handle this problem?

Recursive Type Declarations

```
struct {  
    S_symbol sym;  
    Ty_ty ty;}  
Ty_name
```

`type list = {first: int, rest: list}`

- **Solution: use two passes**
 - **1. Put all the “headers” in the environment first**
(though they do not have bodies)
 - What is a header in this example? `type list =`
 - Use the special “name” type for the header
 - **2. Call `transTy` on the “body” of the type declaration**
 - That is, the record type expression
`{first: int, rest: list}`
 - The `type` that `transTy` returns can then be assigned into the `ty` field within the `Ty_Name` struct.

Recursive Type Declarations

- Every cycle in a set of mutually recursive type declarations **MUST** pass through a record or array declaration !
 - Tiger语言的重要特性之一

`type a = b`
`type b = a`

Tiger

It is illegal in Tiger.

`type a = b`
`type b = {i: a}`

Tiger

`type a = b`
`type b = d`
`type c = a`
`type d = a`

Tiger

It is illegal in Tiger.

contains an illegal cycle:
a -> b -> d -> a

Otherwise, the type checker will not terminate!

Recursive Function Declarations

- **Mutually recursive functions** handled similarly
f call g, g call f
- **Problem:** P when we process the right hand side of function declarations, we may encounter symbols that are not defined in the env yet
- **Solution**
 - First pass: gathers information about the *header* of each function but leaves the bodies of the functions untouched.
 - Second pass: processes the **bodies** of all functions with the augmented environment.